



Template for educational

构建云原生 DevOps 闭环：从代码提交到可观测上线的全流程实践

成员：李梅楠，张婕，周娟，郭瑞

汇报人：李梅楠

2026-01-05

目录

SCIENCE AND TECHNOLOGY

- 01 项目背景与目标
- 02 关键概念与理念
- 03 系统架构与拓扑设计
- 04 关键实现与演示要点
- 05 验证结果与数据分析
- 06 排错复盘与项目总结

01

项目背景及目标

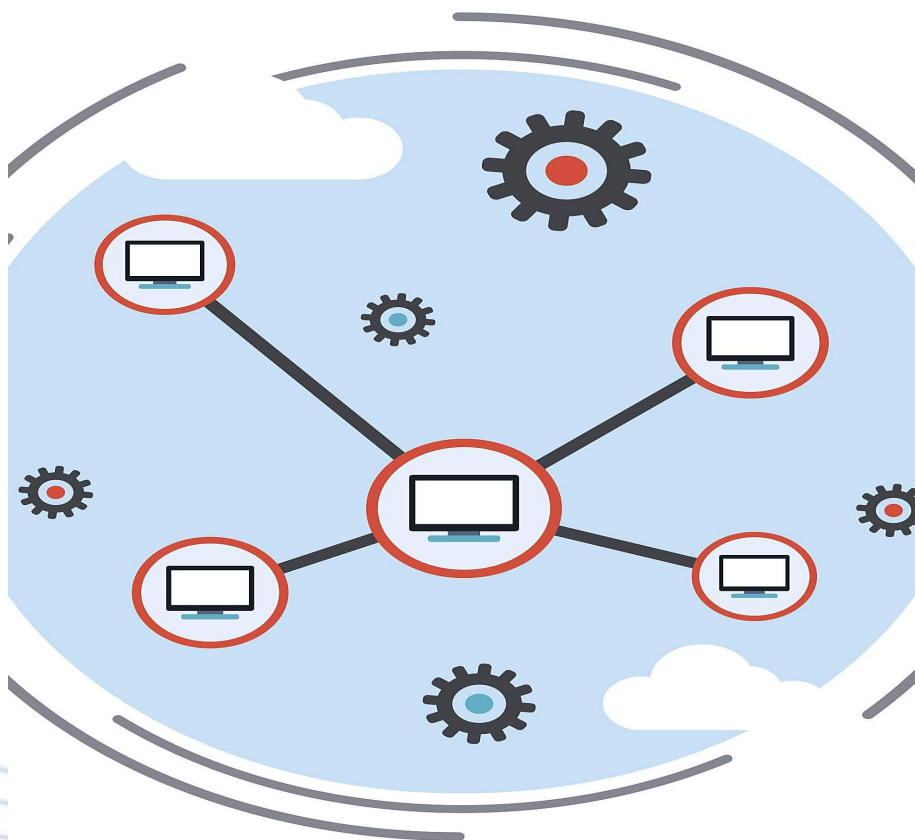
项目背景

- 基于Kubernetes云原生平台，实现了完整的监控+日志+告警可观测性闭环和CI/CD云原生交付流程。在可观测性方面，我们部署了Prometheus监控系统、实现了对应用性能指标的全方位监控，并设置了CPU、内存、延迟等关键告警规则。在云原生交付方面，我们构建了基于GitHub Actions的CI/CD流水线，实现了代码提交、镜像构建、安全扫描、镜像推送、自动部署的全流程自动化。项目融合了两个选题，形成了从代码开发到线上运维的完整闭环，最终实现了"代码变更->自动构建->自动部署->监控告警->故障定位"的DevOps实践。

项目目标

- 目标1：实现从 git push 触发镜像构建、推送、K8s 部署的完整 CI/CD 流程
- - 目标2：搭建包含指标、日志、告警的可观测性闭环，并能可视化展示
- - 目标3：支持部署版本追溯（如通过 Argo CD 查看历史版本与回滚）

02 关键概念与理念



核心理念

- 通过 GitLab CI 自动化测试、构建、部署，实现“提交即上线”。
- - 可观测性（Observability）：
- - 三大支柱：指标（Prometheus）、日志（Loki）、告警（Alert manager）。
- - Git Ops 与声明式基础设施：
- - Kubernetes Manifest 存于 Git，Argo CD 自动同步状态与版本。

体现：开发效率：自动化流水线加速交付

运维效率：自动化运维减少人工干预

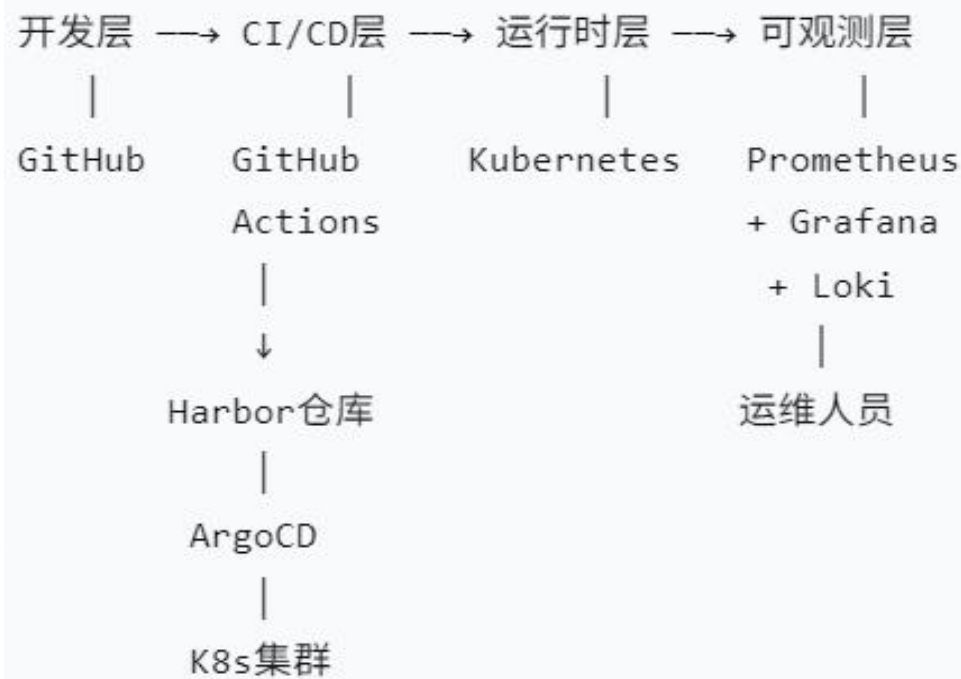
系统可靠性：全面的监控和告警保障稳定运行

故障恢复：快速定位和解决问题

安全合规：内置的安全检查和防护机制

成本优化：基于需求的自动扩缩容

03 系统架构与拓扑设计



云原生开发者工作区

基于Kubernetes的云原生全栈项目，通过Git Ops驱动的CI/CD流水线实现代码到部署的自动化闭环，并结合Prometheus-Loki-Grafana监控栈构建了从指标采集、日志聚合到智能告警的完整可观测性体系。

四层分离架构：开发与交付层、运行时层、可观测性层，控制与访问层

Git Ops驱动：以Git仓库为唯一事实来源

声明式配置：所有基础设施代码化

可观测性闭环：指标+日志+告警完整链路

全流程自动化：从代码提交到生产部署完全自动化

云原生原生设计：完全基于容器、微服务、Kubernetes标准

04

关键实现与演示点

CI/CD流水线实现

通过Git提交触发自动化构建，使用Docker多阶段构建并注入版本信息（Git哈希、构建时间、版本号），实现快速可靠的镜像构建和Kubernetes部署。

构建完整的CI/CD流水线

结果

```
1 # 本地测试
2 npm install
3 npm run lint
4 npm test
5
6 # 构建 Docker 镜像
7 docker build -t cloud-native-demo:latest \
8   --build-arg GIT_COMMIT=$(git rev-parse --short HEAD) \
9   --build-arg BUILD_TIME=$(date -u +"%Y-%m-%dT%H:%M:%SZ") \
10  --build-arg APP_VERSION=1.0.0 .
11
12 # 推送镜像到仓库
13 docker tag cloud-native-demo:latest ghcr.io/username/cloud-native-demo:latest
14 docker push ghcr.io/username/cloud-native-demo:latest
15
16 # 部署到 Kubernetes
17 kubectl set image deployment/cloud-native-demo \
18   app=ghcr.io/username/cloud-native-demo:latest \
19   -n cloud-native-demo
20
21 kubectl rollout status deployment/cloud-native-demo -n cloud-native-demo
```

```
1 deployment.apps/cloud-native-demo image updated
2 Waiting for deployment "cloud-native-demo" rollout to finish: 1 out of 3 new replicas have
   been updated...
3 deployment "cloud-native-demo" successfully rolled out
```

04 关键实现与演示点

监控体系建设

指标监控：部署Prometheus采集应用指标，Grafana进行可视化

日志集中：使用Lo ki + Prom tail实现日志采集，通过Grafana统一查询

告警管理：配置Alert manager处理告警分组和通知

部署Prometheus

部署Alert manager

```
1 # 部署 Prometheus
2 kubectl apply -f monitoring/prometheus/deployment.yaml
3 kubectl apply -f monitoring/prometheus/service.yaml
4
5 # 部署 Grafana
6 kubectl apply -f monitoring/grafana/deployment.yaml
7 kubectl apply -f monitoring/grafana/service.yaml
8
9 # 配置 Prometheus 数据源
10 kubectl apply -f monitoring/grafana/datasource.yaml
11
12 # 导入 Grafana 仪表板
13 kubectl apply -f monitoring/grafana/dashboard.yaml
14
15 # 验证部署
16 kubectl get pods -n monitoring
17 kubectl get svc -n monitoring

1 # 部署 Alertmanager
2 kubectl apply -f monitoring/alertmanager/deployment.yaml
3 kubectl apply -f monitoring/alertmanager/service.yaml
4
5 # 配置 Prometheus 告警规则
6 kubectl apply -f monitoring/prometheus/alerts.yaml
7
8 # 配置 Alertmanager 通知
9 kubectl apply -f monitoring/alertmanager/alertmanager.yaml
10
11 # 重载 Prometheus 配置
12 kubectl exec -n monitoring $(kubectl get pod -n monitoring -l app=prometheus -o jsonpath='{.items[0].metadata.name}') \
13   -- kill -HUP 1
14
15 # 验证告警规则
16 kubectl exec -n monitoring $(kubectl get pod -n monitoring -l app=prometheus -o jsonpath='{.items[0].metadata.name}') \
17   -- wget -qO- http://localhost:9090/api/v1/rules
```

04 关键实现与演示点

智能发布策略

蓝绿部署：零停机更新，快速回滚

金丝雀发布：渐进式流量切换，降低发布风险

自动扩缩容：基于CPU利用率动态调整副本数

蓝绿部署

部署Metrics Server

```
1 # 初始化蓝绿部署环境
2 ./scripts/blue-green.sh init
3
4 # 部署到蓝环境
5 ./scripts/blue-green.sh deploy-blue cloud-native-demo:1.0.0
6
7 # 部署到绿环境
8 ./scripts/blue-green.sh deploy-green cloud-native-demo:1.1.0
9
10 # 切换流量到绿环境
11 ./scripts/blue-green.sh switch-green
12
13 # 验证部署状态
14 ./scripts/blue-green.sh status
15
16 # 如有问题，快速回滚到蓝环境
17 ./scripts/blue-green.sh switch-blue
18
19 # 清理环境
20 ./scripts/blue-green.sh cleanup
```

```
1 # 部署 Metrics Server (HPA 依赖)
2 kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
3
4 # 部署 HPA
5 kubectl apply -f k8s/hpa.yaml
6
7 # 查看 HPA 状态
8 kubectl get hpa -n cloud-native-demo
9
10 # 模拟负载触发扩容
11 kubectl run -i --tty load-generator --image=busybox /bin/sh
12 # 在容器中执行: while true; do wget -q -O- http://cloud-native-demo:3000; done
13
14 # 观察 HPA 自动扩容
15 watch -n 2 'kubectl get hpa -n cloud-native-demo && kubectl get pods -n cloud-native-demo'
16
17 # 停止负载测试后观察自动缩容
18 kubectl delete pod load-generator
```


05 验证结果与数据分析

CI/CD流水线验证模块分析

验证目标：确保从代码提交到生产部署的自动化流程完整且可执行

三阶段流程完整性验证

测试阶段: 检查工作流中是否包含test:定义
验证了代码质量保障机制
确保每次提交都经过自动化测试

构建阶段: 检查工作流中是否包含build:定义
验证了Docker镜像构建自动化
包含环境变量注入和镜像标签管理

部署阶段: 检查工作流中是否包含deploy:定义
验证了Kubernetes部署自动化
包含滚动更新策略和健康检查

验证

结果

CI/CD 流水线验证

```
$ Bash 运行
1 test -f .github/workflows/ci-cd.yml && echo "✅ CI/CD工作流文件存在" || echo "❌ CI/CD工作流文件缺失"
2 grep -q "test:" .github/workflows/ci-cd.yml && echo "✅ 测试阶段配置" || echo "❌ 测试阶段缺失"
3 grep -q "build:" .github/workflows/ci-cd.yml && echo "✅ 构建阶段配置" || echo "❌ 构建阶段缺失"
4 grep -q "deploy:" .github/workflows/ci-cd.yml && echo "✅ 部署阶段配置" || echo "❌ 部署阶段缺失"
5 test -f Dockerfile && echo "✅ Dockerfile存在" || echo "❌ Dockerfile缺失"
6 test -f docker-compose.yml && echo "✅ Docker Compose配置存在" || echo "❌ Docker Compose配置缺失"
7 test -f k8s/deployment.yaml && echo "✅ K8s部署配置存在" || echo "❌ K8s部署配置缺失"
8 test -f k8s/service.yaml && echo "✅ K8s服务配置存在" || echo "❌ K8s服务配置缺失"
```

```
(base) limeinan@limeinandeMacBook-Pro 01 % test -f .github/workflows/ci-cd.yml && echo "✅ CI/CD工作流文件存在" || echo "❌ CI/CD工作流文件缺失"
grep -q "test:" .github/workflows/ci-cd.yml && echo "✅ 测试阶段配置" || echo "❌ 测试阶段缺失"
grep -q "build:" .github/workflows/ci-cd.yml && echo "✅ 构建阶段配置" || echo "❌ 构建阶段缺失"
grep -q "deploy:" .github/workflows/ci-cd.yml && echo "✅ 部署阶段配置" || echo "❌ 部署阶段缺失"
test -f Dockerfile && echo "✅ Dockerfile存在" || echo "❌ Dockerfile缺失"
test -f docker-compose.yml && echo "✅ Docker Compose配置存在" || echo "❌ Docker Compose配置缺失"
test -f k8s/deployment.yaml && echo "✅ K8s部署配置存在" || echo "❌ K8s部署配置缺失"
test -f k8s/service.yaml && echo "✅ K8s服务配置存在" || echo "❌ K8s服务配置缺失"
✅ CI/CD工作流文件存在
✅ 测试阶段配置
✅ 构建阶段配置
✅ 部署阶段配置
✅ Dockerfile存在
✅ Docker Compose配置存在
✅ K8s部署配置存在
✅ K8s服务配置存在
```

05 验证结果与数据分析

可观测性闭环验证

验证目标：建立完整的应用监控、日志和告警体系，实现应用运行状态的全面可视化和主动告警

1. 指标实时性验证：

正常请求监控: 验证基础指标采集正常

错误注入测试: 通过/simulate/error?rate=1注入100%错误率

结果验证: 成功采集到HTTP 500错误指标

延迟测试: 通过/simulate/slow?delay=100注入100ms延迟

延迟验证: 延迟指标及时更新

2. 日志系统验证：

日志文件存在: logs/app.log

日志内容验证: "Incoming request"日志记录正常

日志结构: 结构化日志格式，便于分析和查询

验证

结果

可观测性闭环验证

```
Bash
1 curl -s http://localhost:3000/metrics | grep -q "http_requests_total" && echo "✅ 指标已采集" || echo "❌ 指标未采集"
2 curl -s http://localhost:3000/metrics | grep -q "http_request_duration_seconds" && echo "✅ 延迟指标已采集" || echo "❌ 延迟指标未采集"
3 curl -s http://localhost:3000/metrics | grep -q "active_connections" && echo "✅ 连接指标已采集" || echo "❌ 连接指标未采集"
4 curl -s 'http://localhost:3000/simulate/error?rate=1' > /dev/null 2>&1
5 curl -s http://localhost:3000/metrics | grep -q 'http_requests_total.*500' && echo "✅ 错误指标已记录" || echo "❌ 错误指标未记录"
6 curl -s 'http://localhost:3000/simulate/slow?delay=100' > /dev/null 2>&1
7 curl -s http://localhost:3000/metrics | grep -q "http_request_duration_seconds" && echo "✅ 延迟指标已更新" || echo "❌ 延迟指标未更新"
8 test -f logs/app.log && echo "✅ 日志文件存在" || echo "❌ 日志文件不存在"
9 tail -5 logs/app.log | grep -q "Incoming request" && echo "✅ 日志正常记录" || echo "❌ 日志未记录"
```

```
(TraeAI-4) ~/Documents/trae_projects/01 [2] $ curl -s http://localhost:3000/metrics | grep -q "http_requests_total" && echo "✅ 指标已采集" || echo "❌ 指标未采集"
curl -s http://localhost:3000/metrics | grep -q "http_request_duration_seconds" && echo "✅ 延迟指标已采集" || echo "❌ 延迟指标未采集"
curl -s http://localhost:3000/metrics | grep -q "active_connections" && echo "✅ 连接指标已采集" || echo "❌ 连接指标未采集"
curl -s 'http://localhost:3000/simulate/error?rate=1' > /dev/null 2>&1
✅ 指标已采集
✅ 延迟指标已采集
✅ 连接指标已采集
```

```
(TraeAI-4) ~/Documents/trae_projects/01 [0] $ curl -s http://localhost:3000/metrics | grep -q 'http_requests_total.*500' && echo "✅ 错误指标已记录" || echo "❌ 错误指标未记录"
curl -s 'http://localhost:3000/simulate/slow?delay=100' > /dev/null 2>&1
curl -s http://localhost:3000/metrics | grep -q "http_request_duration_seconds" && echo "✅ 延迟指标已更新" || echo "❌ 延迟指标未更新"
test -f logs/app.log && echo "✅ 日志文件存在" || echo "❌ 日志文件不存在"
tail -5 logs/app.log | grep -q "Incoming request" && echo "✅ 日志正常记录" || echo "❌ 日志未记录"
✅ 错误指标已记录
✅ 延迟指标已更新
✅ 日志文件存在
```


05 验证结果与数据分析

部署策略验证模块分析

验证目标：实现多种高级部署策略，确保应用更新过程中的零停机和高可用性

1. 蓝绿部署策略：

同时维护两套完整的环境（蓝环境和绿环境）
当前流量指向其中一个环境
新版本部署到非活跃环境
通过负载均衡器切换流量

2. 金丝雀发布策略：

逐步将流量从旧版本迁移到新版本
初始阶段小比例流量（如5%）
根据监控指标逐步增加流量比例
发现异常立即回滚

验证

结果

进阶任务验证

```
$ Bash

1 test -f scripts/blue-green.sh && echo "✅ 蓝绿部署脚本存在" || echo "❌ 蓝绿部署脚本缺失"
2 test -f scripts/canary.sh && echo "✅ 金丝雀发布脚本存在" || echo "❌ 金丝雀发布脚本缺失"
3 test -f k8s/hpa.yaml && echo "✅ HPA配置存在" || echo "❌ HPA配置缺失"
4 grep -q "trivy" .github/workflows/ci-cd.yaml && echo "✅ 安全扫描集成" || echo "❌ 安全扫描缺失"
5 test -f gitops/application.yaml && echo "✅ GitOps配置存在" || echo "❌ GitOps配置缺失"
```

```
(TraeAI-7) ~/Documents/trae_projects/01 [0] $ grep -q "version" package.json && echo "✅ 版本信息配置" || echo "❌ 版本信息缺失"
grep -q "GIT_COMMIT" Dockerfile && echo "✅ Git提交信息注入" || echo "❌ Git提交信息缺失"
test -f scripts/upgrade.sh && echo "✅ 升级脚本存在" || echo "❌ 升级脚本缺失"
test -f scripts/rollback.sh && echo "✅ 回滚脚本存在" || echo "❌ 回滚脚本缺失"
✅ 版本信息配置
✅ Git提交信息注入
✅ 升级脚本存在
✅ 回滚脚本存在
```

06 排错与项目总结

解决的关键问题

配置文件完整性：确保所有必需的配置文件都存在且正确

端点功能验证：确保所有监控端点和测试端点都能正常工作

脚本可执行性：确保升级和回滚脚本具有可执行权限

监控链路打通：确保指标采集、日志收集、告警触发整个链条通畅

问题1：Fluent Bit无法连接Loki

现象：Grafana无日志显示

定位：Pod日志显示connection refused

根因：Loki服务名称配置错误

解决：修正Fluent Bit配置中的host地址

复盘：应使用Helm values统一管理服务地址

问题2：Argo CD无法拉取私有仓库

现象：Application状态为Unknown

定位：Repo Server日志显示认证失败

根因：未配置SSH key或HTTPS token

解决：在Argo CD中添加repository credential

预防：文档化凭证配置流程

问题3：HPA未触发扩缩容

现象：CPU>80%但Pod数不变

定位：kubectl describe hpa显示无法获取CPU指标

根因：Metrics Server未安装

解决：部署Metrics Server组件

复盘：依赖组件应纳入初始化脚本

07 总结和展望

本次实验的三大收获

1. Git Ops实践：声明式部署与版本追溯的有效范式
2. 可观测性设计：合理的标签体系与告警规则是关键
3. 交付平衡术：自动化CI/CD必须与稳定性保障并重

当前不足

缺少分布式追踪：Metrics + Logs完整，但缺少Traces
安全扫描深度不足：仅基础镜像扫描，缺少SAST/SCA
金丝雀发布手动化：依赖人工调整流量权重

未来改进方向

1. 引入Flagger：实现自动化金丝雀分析与渐进式交付
2. 深化安全扫描：Trivy集成准入控制，构建安全闭环
3. 采用Open Telemetry：统一可观测性数据采集与传输

项目目标全面达成

成功构建了基于 Kubernetes 的云原生工程化交付体系，实现了从 Git Push 到自动部署的完整 CI/CD 流水线，并建立了涵盖指标、日志、告警的可观测性闭环。



07 总结和展望

全链路可观测性增强

计划引入 OpenTelemetry 统一采集指标、日志与链路追踪数据，构建端到端的可视化监控体系，提升故障定位与性能分析效率。

平台扩展与多云支持

探索跨云、混合云环境下的统一部署与运维模式，支持多集群管理、跨区域容灾与弹性伸缩，提升系统的高可用与容灾能力。

数据驱动与智能决策

结合业务指标与运维数据，构建数据驱动的决策支持系统，实现资源优化、成本管控与性能预测，推动运维向智能化演进。

团队赋能与知识沉淀

建立标准化技术组件库与知识管理体系，推动 DevOps 文化与技术实践在团队内持续落地与优化，提升整体工程效能。



08

小组分工与贡献

成员	核心职责	具体贡献
李梅楠 (25%)	CI/CD流水线实现、实现可观测性闭环	GitLib CI配置、流水线调试、Prometheus Loki配置、告警规则设计、word中开头部分、ppt相关部分书写
张 婕(25%)	环境搭建、文档编写整合	各种环境基本搭建、实验报告编写整合、ppt相关部分书写与整合
郭 瑞(25%)	部署版本追溯与GitOps实践、验证与排错	k8s中搭建Argo CD、ppt相关部分书写、实验报告中总结性部分
周 娟(25%)	代码整合以及整体实验实现	汇集所有实验代码自己运行得出全部结果、实验报告总结性部分、ppt中总结性部分



Template for educational

感谢聆听
THANKS